CSCI4180 Introduction to Cloud Computing and Storage

Ryan Chan

October 19, 2026

Abstract

This is a note for **CSCI4180 Introduction to Cloud Computing and Storage**.
Contents are adapted from the lecture notes of CSCI4180, prepared by Patrick P. C. Lee, as well as some online resources.
This note is intended solely as a study aid. While I have done my best to ensure the accuracy of the content, I do not take responsibility for any errors or inaccuracies that may be present. Please use the material thoughtfully and at your own discretion.
If you believe any part of this content infringes on copyright, feel free to contact me, and I will address it promptly.
Mistakes might be found. So please feel free to point out any mistakes.

Contents

1 Fundamentals of Cloud Computing **2**
1.1 Introduction 2
1.2 Cloud Computing Concepts 3
1.3 Service Delivery Models 5
1.4 Deployment Scenarios 6
1.5 Security Issues 7

Chapter 1

Fundamentals of Cloud Computing

1.1 Introduction

1.1.1 Definition

Cloud computing provides computing resources, such as processing power, storage, networking, and applications, as services over a network. These resources may be physical or virtual and are typically hosted in data centers. IBM emphasizes on-demand access to these resources and usage-based pricing.[1]

In **NIST**'s definition, users obtain configurable resources from a shared pool over a network, with rapid allocation and release and little administrative effort. NIST identifies five essential characteristics:[2]

- **On-demand self-service:** Users provision resources without requiring a provider's staff to process each request.
- **Broad network access:** Services are accessible over networks using standard mechanisms and different types of client devices.
- **Resource pooling:** Providers dynamically allocate shared resources among multiple customers according to demand.
- **Rapid elasticity:** Allocated capacity can expand and contract quickly as demand changes.
- **Measured service:** Resource consumption is metered, monitored, and reported.

Measured service supports usage-based billing, but does not require every service to use the same pricing model. A **service-level agreement (SLA)** specifies service commitments, such as availability; it is distinct from usage metering.

Definitions vary in emphasis. The Berkeley report *Above the Clouds* discusses both applications delivered over the Internet and the data-center hardware and systems software supporting them. In that report, the latter constitute the "cloud."[3]

In general, cloud computing combines hardware and software technologies with automated service delivery to make computing resources available on demand.

1.1.2 Reasons for Using Clouds

Cloud computing can reduce upfront investment in hardware, accelerate software development, testing, and deployment, and provide access to infrastructure that an organization would otherwise have to build and maintain itself. Elastic resource allocation can also improve utilization.

[1] IBM: What Is Cloud Computing?

[2] NIST SP 800-145: The NIST Definition of Cloud Computing. This also defines the three service models and four deployment models discussed below.

[3] Armbrust et al.: Above the Clouds: A Berkeley View of Cloud Computing (2009).

The unit price of on-demand capacity may exceed that of fully utilized owned or reserved capacity. However, releasing resources when they are no longer needed can reduce the total cost. Allocated but idle resources, retained storage, and other services may still incur charges; cost savings are not automatic.

Traditional infrastructure management can involve long procurement and setup times, complex administration, and low utilization when capacity is sized for peak demand. Cloud services can alleviate these problems.

Concerns include service reliability, security, data location and control, interoperability, vendor lock-in, and cost predictability. Differences among provider APIs and service capabilities can complicate migration and integration.

1.1.3 Grid Computing

Grid computing coordinates resources from multiple networked computers, potentially across different organizations, to execute computational workloads. A workload may be split into many jobs that run in parallel; a grid is not restricted to solving one problem at a time.[4]

Cloud computing emphasizes on-demand service delivery, resource pooling, elasticity, and metering. Both approaches use distributed resources, but cloud computing is not simply an extension of grid computing. A grid can also run on cloud infrastructure.

1.2 Cloud Computing Concepts

1.2.1 Elasticity and Scalability

Scalability is a system's ability to handle increased workload by adding resources while maintaining acceptable performance. **Vertical scaling** adds capacity to an existing machine, while **horizontal scaling** adds more machines or instances.

Elasticity is the ability to adjust allocated resources to changing demand, including releasing resources when demand falls. These adjustments are often automated, but they are not necessarily instantaneous.

For example, an application that can handle more traffic when additional servers are added is scalable. If it adds servers during a traffic spike and removes them afterward, it is also elastic.

Providers and customers use scaling policies to meet performance and availability targets while controlling costs. Actual charges depend on the service's billing model and the resources retained or consumed.

1.2.2 Provisioning and Deprovisioning

Provisioning allocates and configures resources for users or applications, such as creating virtual machines, storage volumes, or network connections. Access policies and quotas determine who may request resources and how much they may allocate.

Deprovisioning releases resources or revokes access when they are no longer needed. Releasing a compute instance does not necessarily delete its associated storage or stop all related charges.

Self-service provisioning allows customers to request resources through a portal or API without a lengthy manual process, reducing provisioning delays.

1.2.3 Virtualization

Virtualization creates logical abstractions of physical computing resources so that they can be allocated and managed more flexibly. It can divide one physical resource into several virtual resources or combine multiple physical resources into a logical pool.

[4] AWS: What Is Grid Computing?

Examples include virtual CPUs, virtual disks, and virtual networks. Server virtualization allows multiple virtual machines (VMs), each with its own guest operating system, to share a physical machine. It is a common enabling technology for cloud computing, but cloud services do not have to use VMs.[5]

Virtualization provides:

- **Partitioning:** Physical resources are divided among multiple VMs, allowing several operating systems and applications to run on one machine.
- **Isolation:** VMs have separate execution environments. Isolation reduces interference, but is not absolute: vulnerabilities, misconfiguration, and resource contention can still affect other VMs or the host.
- **Encapsulation:** A VM's configuration and virtual disks can be represented as files; snapshots may also capture memory and execution state. Moving a VM still requires compatible virtualization support.

Virtualization can consolidate workloads, improve hardware utilization, and make resource allocation more flexible.

A **hypervisor** creates and manages VMs and mediates their access to CPU time, memory, storage, and networking. It may run directly on hardware or above a host operating system.

Unlike a VM, a **container** normally shares the kernel of its host environment, which may itself be a VM. A container packages an application with its required user-space dependencies, rather than a separate guest kernel. Containers provide process isolation, but their shared kernel creates a different security boundary from that of VMs.[6]

1.2.4 Multitenancy

Multitenancy allows multiple customers or organizations, called *tenants*, to share some part of a system while keeping their data and access permissions logically separated. Sharing can occur at the infrastructure, platform, or application layer.

For SaaS, deployment arrangements can range from separate application instances to a shared application instance:

- **Separate instances:** Each customer has a separate application instance and possibly a separate database, while the underlying infrastructure may be shared. This is single-tenancy at the application-instance level, even if the infrastructure is multi-tenant.
- **Shared instance:** Multiple customers use the same application instance, with tenant-aware authorization and data separation. Databases may be shared or separated.

For PaaS, customers deploy their own applications onto a shared platform. Isolation may use VMs, containers, runtime sandboxes, or other mechanisms; it does not require one VM per customer.

Key challenges include preventing cross-tenant data access, enforcing resource limits, and avoiding excessive performance interference between tenants.

1.2.5 Application Programming Interface (API)

An **application programming interface (API)** defines the operations and data formats through which software components interact. Cloud APIs allow applications and management tools to provision resources, configure services, and access data.

Although common protocols and standards exist, providers often expose different APIs, service semantics, and features. This can create **vendor lock-in**: dependence on a provider that makes migration difficult or expensive.

[5] IBM: What Is Virtualization?
[6] Docker documentation: What Is a Container?

Standard interfaces, portable data formats, and abstraction layers can reduce lock-in, but do not eliminate it. Applications may still depend on provider-specific behavior or managed services.

1.3 Service Delivery Models

The three principal cloud service models differ in what the provider manages and what remains under the customer's control.[7]

1.3.1 Infrastructure as a Service (IaaS)

Infrastructure as a Service (IaaS) supplies compute, storage, and networking resources. The provider operates the physical infrastructure; customers typically manage their guest operating systems, applications, and data.

Customers can provision and release resources on demand. Virtualization is common, although physical servers may also be offered. Examples of infrastructure services include Amazon EC2 for compute and Amazon S3 or Azure Blob Storage for object storage.

1.3.2 Platform as a Service (PaaS)

Platform as a Service (PaaS) provides a managed environment for developing, testing, deploying, and running applications. The provider manages the underlying infrastructure and platform software, while customers manage their applications and data.

Platform capabilities may include language runtimes, application servers, databases, messaging, and development tools. Customers may still configure instance sizes, scaling limits, or service tiers, even though they do not manage the physical hardware. Examples include Google App Engine and Heroku.

1.3.3 Software as a Service (SaaS)

Software as a Service (SaaS) delivers a provider-managed application over a network. The provider maintains the application and its underlying platform and infrastructure. Customers use and configure the application and remain responsible for appropriate access and data handling. Examples include Google Drive, iCloud, and Dropbox.

Other service labels include Storage as a Service, Data as a Service, Testing as a Service, and Integration as a Service. These may overlap with the three principal models.

1.3.4 Serverless Computing

Serverless computing lets developers use managed execution environments and backend services without provisioning or administering the underlying servers. Servers still exist; the provider manages them. Scaling and billing behavior depend on the particular service.[8]

Two commonly associated models are:

- **Function as a Service (FaaS):** Customers deploy functions that run in response to requests or events, while the provider manages their execution infrastructure. Examples include AWS Lambda and Azure Functions.
- **Backend as a Service (BaaS):** Applications access managed backend capabilities, such as authentication, databases, and storage, through APIs or SDKs. BaaS is a service model, not a serverless development framework.

[7] IBM: IaaS, PaaS, SaaS: What's the Difference?
[8] AWS: Serverless Computing.

Serverless applications may combine FaaS and BaaS, but serverless computing is broader than a fixed combination of the two; it also includes other managed execution and data services.

1.4 Deployment Scenarios

NIST identifies four cloud deployment models: public, private, hybrid, and community. A virtual private cloud is a related networking concept, rather than a fifth NIST deployment model.

1.4.1 Public Cloud

A **public cloud** offers services to the general public. Customers usually access it over the Internet, although private connections may be available. Infrastructure is commonly shared, with logical isolation between customers. Services, eligibility conditions, and pricing vary by provider.

1.4.2 Private Cloud

A **private cloud** serves one organization exclusively. It may be operated by that organization, a third party, or both, and may be hosted on or off premises.

Private clouds offer greater control over infrastructure and policies, but are not inherently more secure or reliable. They still face capacity limits, network failures, misconfiguration, and security threats. Their effectiveness depends on design and operation.

1.4.3 Hybrid Cloud

A **hybrid cloud** connects distinct cloud environments while allowing data or applications to move between them. For example, an organization may use a private cloud for routine workloads and a public cloud for additional capacity during demand peaks.

Hybrid deployments require suitable connectivity, identity and access controls, monitoring, and management. Merely using several disconnected clouds does not establish a hybrid cloud.

1.4.4 Community Cloud

A **community cloud** serves organizations with shared requirements, such as a common mission or security policy. Participating organizations or a third party may operate it. Ownership and cost allocation depend on the arrangement; it need not be purchased by a single customer.

1.4.5 Virtual Private Cloud (VPC)

A **virtual private cloud (VPC)** is a logically isolated virtual network within a public cloud. Customers configure network elements such as address ranges, subnets, routing, and access controls. A VPC does not by itself imply dedicated physical hardware or a private-cloud deployment.[9]

A VPC can connect to an organization's existing network through a VPN or a dedicated connection, allowing cloud resources to extend that network.

When choosing a deployment model, organizations should consider cost, control, workload characteristics, connectivity, and shared requirements. Public clouds reduce the need to own infrastructure; private and community clouds require capacity planning and operation; hybrid clouds add flexibility but also integration complexity.

[9] AWS documentation: What Is Amazon VPC?

1.5 Security Issues

Cloud security follows a **shared responsibility model**. Providers protect the infrastructure and service components they operate. Customers remain responsible for the components they control, including account permissions, service configuration, and data handling. The exact division varies across IaaS, PaaS, SaaS, and individual services.[10]

Customers may have limited visibility into a provider's internal infrastructure. They must assess available security documentation, audit evidence, logs, and contractual commitments rather than assume that outsourcing infrastructure transfers all security responsibility.

- **Isolation failures:** Vulnerabilities or misconfiguration in hypervisors, kernels, or shared services may allow unauthorized access across tenants or to the host. Resource contention can also affect availability.
- **Insecure APIs:** Weak authentication, excessive permissions, exposed credentials, or implementation flaws can allow unauthorized operations. Different provider APIs increase management complexity, but lack of standardization alone does not make an API insecure.
- **Data loss and leakage:** Accidental deletion, corruption, misconfiguration, compromised accounts, and service failures can threaten data. Cloud storage does not remove the need to define backup, recovery, retention, encryption, and access-control responsibilities. Replication alone is not a substitute for backups.
- **Account or service hijacking:** Stolen passwords, tokens, or API keys may allow an attacker to read or alter data, change services, redirect traffic, or consume resources. Least-privilege access, multi-factor authentication where applicable, credential protection, and monitoring help reduce this risk.

[10] AWS: Shared Responsibility Model.